PANW QA Interview — Sr Staff Engineer Take-Home Assignment

Congratulations on advancing to the next stage of our interview process!

As a Sr Staff candidate, you're expected to go beyond writing tests — you design frameworks, enforce quality gates, build CI pipelines, and think at team scale. This take-home assignment will give you an opportunity to demonstrate these skills.

You have **6–8 hours** to complete this project. Your submission must include a functional test framework, passing tests, a generated report, and a CI pipeline definition that ties everything together.

Assignment — Multi-Environment Data Consistency (REST Countries + Open-Meteo)

Targets: https://restcountries.com/v3.1 (free, no auth) + https://api.open-meteo.com/v1/forecast (free, no auth)

Unique angle: The same test logic runs against two different "environments" (two independent APIs). Environment configuration — base URLs and thresholds — lives entirely in YAML. Tests are API-agnostic.

Task

1. Create `config/environments.yaml` with two environments:
   - `countries` : base URL `https://restcountries.com/v3.1` , `max_response_time: 2.0` , `min_results_count: 1`
   - `weather` : base URL `https://api.open-meteo.com/v1` , `max_response_time: 3.0` , `min_results_count: 1`
2. Build an `environment` fixture that reads the YAML and injects `base_url` and `thresholds` into tests based on a `--env` CLI flag.

3. Countries tests:
    - `GET /region/europe` — assert result count > 40
    - `GET /name/germany` — validate schema: `name`, `capital`, `population`, `currencies`, `languages` all present
    - `GET /all?fields=name,population` — assert every country has population > 0
    - Cross-reference: a country found via `/name` search must also appear in `/region` results
4. Weather tests: parametrize 5 cities from `test_data/cities.json`, call the forecast endpoint for each, validate temperature range is reasonable (-80 to 60°C), validate hourly entry count is > 0, validate `timezone` field is present.
5. Both APIs must respond within the `max_response_time` threshold defined in `environments.yaml` — not hardcoded anywhere in test code.
6. Support `--env countries`, `--env weather`, or no flag (runs both). Implement via `pytest_addoption`.
7. Generate a combined **Allure** report with a separate section per environment.

Requirements

- `pytest`, `requests`, `allure-pytest`, `pyyaml`
- `--env` custom CLI flag
- Environment fixture in top-level `conftest.py`
- `test_data/cities.json` committed to repo

Evaluation Criteria

Area	Weight
Environment abstraction design	30%
YAML-driven config (zero hardcoded values)	25%
Test logic reuse across two APIs	25%
CLI flag implementation	10%
Allure report per-environment sections	10%

CI Pipeline Requirement

Provide a `.github/workflows/ci.yml` or `.gitlab-ci.yml` in your repo that:

- Triggers on push to any branch
- Sets up Python environment and installs dependencies
- Runs the full test suite
- Uploads the test report (HTML or Allure) as a pipeline artifact
- Fails the pipeline if any test fails or if the quality gate threshold is breached
- (Bonus) Shows test summary in the pipeline job output

Claude Code Usage (Required)

> ⚠️ This section is **mandatory**. Submissions without Claude Code evidence will be marked incomplete.

What to Include in Your Repo

1. **`.claude/rules/` folder — minimum 3 rule files:**

   - `testing-standards.md` — project testing conventions (e.g. "parametrize from JSON always, never inline test data, every endpoint needs a schema validation test")
   - `code-style.md` — Python style rules specific to this framework (e.g. "all validators in src/validators/, never inline assertions, use type hints")
   - `framework-rules.md` — architecture constraints (e.g. "reporters must extend BaseReporter", "test files must not import from other test files", "all configs in config/ — never hardcoded")

> Rules must reference **this specific framework's architecture** — not generic Python style.

2. **`.claude/skills/` folder — minimum 2 skill files:**

- `test-generator.md` — given endpoint URL + method + response fields, generates a complete pytest test file with fixtures, parametrize, markers, positive + negative tests
- `validator-generator.md` — given a sample JSON response, generates a typed validator/schema class with per-field type checks and required field validation

3. **`CLAUDE_LOG.md` — session log documenting:**

- [ ] At least **2 tasks run with parallel agents** — describe what ran in parallel, why those tasks were independent, and the time saved
- [ ] One **architectural decision validated with Claude** — what was the decision, what did Claude suggest, did you follow it or override it and why?
- [ ] One case where **Claude's suggestion was wrong for this codebase** — specifically what was wrong and why (e.g. "Claude suggested a singleton pattern but it broke test isolation")
- [ ] How your **rules changed Claude's output** — concrete before/after example

4. Use Claude for at least these tasks (document each in CLAUDE_LOG.md):

- [ ] Use Claude to generate the framework skeleton, then architect the final version yourself
- [ ] Use parallel agents for at least 2 independent workstreams (e.g. generate API tests + generate schema validators simultaneously)
- [ ] Use Claude to identify edge cases — document which were valid, which were hallucinated
- [ ] Use Claude to review your framework for extensibility gaps — then act on the feedback

---

Submission Instructions

Please create a new GitHub repository containing your complete solution and share the link with us. Your repository should include:

1. All source code (test framework, fixtures, validators, etc.)
2. Configuration files (`config/environments.yaml` , `test_data/cities.json`)
3. Claude Code artifacts (`.claude/rules/` , `.claude/skills/` , `CLAUDE_LOG.md`)
4. CI pipeline configuration (`.github/workflows/ci.yml` or `.gitlab-ci.yml`)

5. A `README.md` with:
    - Setup instructions
    - How to run the tests locally
    - How to interpret the test results
    - Any assumptions or design decisions you made

Questions?

If you have any questions about the assignment, please don't hesitate to reach out to your recruiting contact. We're here to help ensure you have everything you need to succeed.

Good luck! We're excited to see what you build.